Evidence Supplement: The Log Is the Agent

Yohei Nakajima
Untapped Capital, Bellevue, Washington, USA
yohei@untapped.vc
ORCID: 0009-0009-1293-6053

November 2, 2026

This supplement accompanies *The Log Is the Agent: Engineering Inspectable Agent Runs*. It records a bounded, offline check, not a new benchmark study or independent replication. Author metadata and funding/interests are included in the accompanying article. The complete machine-readable inventory is in `manifest.json`; the observed outputs are in `evidence/identity-checked/verification.json` and the accompanying JSONL/projection files.

Pinned implementation and public-v1 separation

The runtime baseline is commit `7070112` (full commit recorded in the manifest) of `https://github.com/yoheinakajima/activegraph`. Source package metadata is 1.13.0; the inspected changelog still calls it unreleased, and a remote release tag is a different identity. This experiment does not install an unqualified latest release and does not establish PyPI publication status. The observed environment used Python 3.11.15 and Pydantic 2.14.2, with the exact local source installed in an isolated environment. Before executing fixtures, the script resolves the imported package location and checks the SHA-256 values and exact inventory of all 110 pinned runtime/fixture files. Any mismatch fails closed. The baseline label alone is not treated as evidence of identity.

The May 2026 arXiv v1, its source archive and extracted editable source are retained unchanged under `archive/arxiv-v1/`. The original PDF hash begins `8e8ba7df94f27f0a`; the original TeX hash begins `b8fb447bad03b55c`. Full SHA-256 values are in the manifest. The public submission timestamp is May 21, 2026; the PDF displays May 22 because the source used a dynamic date. This packet creates a distinct current-baseline article, not an arXiv replacement or a retroactive change to the original experiment.

Exactly what was executed

The script `verification/verify.py` runs two sequential instances of the bundled diligence fixture in a disposable directory. It uses three synthetic companies, `RecordedDiligenceProvider`, a fixed run identifier, seed zero and a frozen event clock. Settings match the inspected fixture-mode quickstart: model-label `claude-sonnet-4-5`, at most five documents per company, at most ten

claims per document, review threshold 0.7, and eight to twelve questions. The label identifies fixture requests; no provider SDK or credential is needed.

One cumulative runtime budget spans all three goals: at most 80 model-behavior invocations, 100 tool calls and USD 5.00 of fixture accounting. The counters do not reset between goals; the wall-clock start does. The final receipt records 55 model-behavior invocations, 48 tool calls and USD 0.2638 of fixture accounting. The 103 model-request events comprise 24 three-turn document-researcher invocations (72 requests) and 31 single-turn invocations. These bookkeeping costs are not provider spend. The guard patches socket connect, connect_ex, bind, create_connection and getaddrinfo; it does not intercept every socket API (for example, UDP sendto or an unbound listen). Zero attempts were observed at the patched entry points. Zero live provider calls is inferred from the fixed fixture provider and guard receipt, not independently metered. The check does not start services or touch the normal quickstart's fixed temporary database. State is created only in the requested disposable output area; no production database is loaded.

The native fixture execution yields the following counts, agreeing with the earlier reported mechanism scale. This agreement does not establish the identity of the May run.

Recorded quantity	Count
Synthetic companies	3
Events	671
Projected objects	93
Projected relations	76
Model-request events served by fixtures	103
Tool-request events	48
Attempts at intercepted socket APIs	0
New live provider calls (inferred)	0

The 93 objects comprise three companies, 24 questions, nine documents, 25 claims, 25 evidence items, one contradiction, three risks and three memos. These are fixture outputs, not validated findings about actual companies. The evidence JSON also gives complete event-type and object-type inventories.

Equality fields and a negative result

Every event is serialized as UTF-8 JSON with keys sorted, compact separators and a trailing newline, in recorded order. The serialization includes id, type, payload, actor, frame id, causing-event id and timestamp. Object projections include id, type, data, version and provenance; relation projections include their complete serialized fields. Ordered arrays are preserved, not sorted to hide ordering differences.

Raw event byte equality fails. In the final captured pair, 48 events differ, exclusively in `payload.latency_seconds` on `tool.responded` events. These are measured local tool timings despite the frozen event clock. This primary pair is identified by `evidence/identity-checked/verification.json`. The earlier pair in `evidence/verification.json` has 47 differing timings (one coincides); both pairs retain their raw hashes and share the same normalized/projection hashes. The count can vary; the differing field class is the invariant checked. The receipt retains both raw

SHA-256 hashes and every differing event id/type/field. The verification code rejects any additional differing field or event-count difference.

A separate normalized event comparison removes only that timing field from tool-response events. The remaining serialized event bytes match. This normalization was defined after the raw timing mismatch was observed; it is a post-hoc engineering check, not a preregistered equality criterion. The full projected object/relation bytes match without this event normalization. Loading the first saved run projects exactly the same object/relation bytes. These observations support equality for these fixed inputs and this pinned implementation. They are not universal byte-determinism guarantees.

The normalized event hash begins `d164a7d9ffb824a2`; the projection hash begins `f444cb69a4b2513a`. Re-executing measured timings can change raw hashes, so reproducing the receipt means checking the stated invariants and fields, not expecting the newly measured raw event file to have an old timing hash. The manifest verifies the exact retained receipt files.

Branch check and execution-verifier boundary

The first run is forked at its last, completed event using the SQLite runtime fork with model and tool caches enabled. The script compares the copied inclusive prefix with the parent's event records, adds one `article_note` object to the child and verifies that the parent's in-memory serialized event list is unchanged. The persisted parent is not reloaded after that edit. The child has exactly one additional object. This copies the whole run through its final event, not an earlier prefix. It does not test arbitrary in-flight branches, promotion-block cuts, fork-and-rerun interventions or external effects. Cache reuse for changed model calls is not measured.

A separate small behavior check illustrates strict verification's boundary. A behavior creates a note whose text comes from mutable test-local state. Changing only the text while preserving event types is accepted by `Runtime.load(..., replay_strict=True)`. Changing the behavior to emit an additional object event is rejected with a replay-divergence error. The check intentionally reports payload-only drift as accepted. No generic verifier is replaced or strengthened by this script.

The inspected implementation projects a fork's copied prefix; it does not invoke the strict behavior verifier as part of the runtime fork. Source inspection shows that model/tool caches can be populated from the parent's full recorded history; this is not demonstrated by the final-cutoff annotation check. Therefore a branch-comparison design must explicitly state whether post-cut parent responses are available. The source also sets a new child's budget to unbounded and seed to zero, defaults tools to the global registry unless supplied, and requeues unfinished work. Review child configuration before continuation. Raw event/projected equality, command/type validation and fresh suffix execution remain different contracts.

Effect-free strict-verification probe

The separate `verification/check_strict_effects.py` uses an in-memory counting tool and ordinary behavior, a fixed two-response fake model provider, and a transient local SQLite log. It verifies all 110 imported package files against the same pin. Initial capture calls the tool once, the ordinary behavior once and the fake provider twice. One passing strict load raises the tool and behavior counts to two while the fake-provider count stays two; the two tool-context UUID keys differ.

No real external I/O or provider is invoked by the mock bodies; no patched network operation was attempted. This confirms that selected-invariant verification can reexecute tool bodies even while model responses come from cache. Unlogged I/O in behavior code can likewise recur, but was not exercised as a real effect. A dry-run or effect-free adapter boundary is required before using verification on live-effect behaviors. The receipt is `evidence/strict-effects/strict-effects.json`.

Reproduction and dependencies

Use the exact runtime and evidence/code revisions and environment versions. The immutable evidence/code snapshot is: `https://github.com/yoheinakajima/activegraph/tree/ba41ac771e6d58b4856acaa8dc8f5d9f624e287a/papers/ieee-software-2026`. The primary receipt is its `evidence/identity-checked/verification.json`. Its SHA-256 is:

```
c6edb8d715bad5115000d6c54a6354cd4c1e430517441e40714d04b3de48b417
```

Runtime source remains byte-identical to 7070112; that baseline alone does not contain this new paper directory. Earlier raw receipts are retained unchanged. The new optimized-Python refusal precedes all output/runtime work; `python -O` is rejected because existing invariant checks use assertions. The static `audit_retained.py` checks pair counts, budget snapshots and the synthetic trace from saved files, without importing the runtime. Full versions are in the manifest. Install the pinned source in a disposable environment, then from the repository root run:

```
python -I papers/ieee-software-2026/verification/verify.py
  --output /absolute/disposable/output
```

The command requires an installed local ActiveGraph package, Click and Pydantic. It neither downloads data nor falls back to an online provider. The execution guard is a fail-closed check on this in-process path, not a general operating-system sandbox. Readers should use a clean process without provider credentials; installation of dependencies is a distinct prior step. The local native-check receipt documents selected replay/fork/model/tool/diligence tests, static typing, docstring coverage and documentation build. Optional database/service tests are not evidence from this packet.

Builds use the already-installed Tectonic engine with cached resources; no TeX runtime was installed. Editable article, supplement and figure source are included. Full SHA-256 values for article PDFs, source, archived public v1, inputs and retained verification outputs are in the packet manifest.

Capture, rights and interpretation limits

The runtime source carries Apache-2.0 and NOTICE requirements. Code, article text, fixture records, public-v1 material and any third-party assets have distinct rights/provenance entries in the rights worksheet; it does not assume the runtime license grants permissions for all future supplements or an author photograph. The fixture documents are bundled synthetic demonstration inputs. Links ending in `fixtures.activegraph.local` are identifiers, not external data-retrieval endpoints.

Neither deterministic projection nor a response cache attests that records were authentic at provider receipt, immutable before the retained snapshot, free of omitted data or scientifically valid. The check cannot evaluate evidence truth, real investment decisions, human oversight quality, privacy compliance, general task accuracy, scale, latency or self-improvement. No additional cross-project replay totals are used. These limits belong in the main article as well as this supplement.

Acknowledgments

This research, including computational resources and model/API usage, was supported by Untapped Capital. The author is its founder and general partner and ActiveGraph's creator and maintainer. OpenAI Codex (GPT-6.1 Sol for the preparation worker) generated and revised all supplement sections and its verification code. Current author-side Claude Opus 5.5 Extra and Gemini Pro reviews informed selected corrections after source/data checks; their exact backend revisions were not authenticated. The article and `tool-use-audit.md` distinguish earlier runtime assistance, unknown original writing tools, fixture-model labels and observed checks. No independent human replication or personal repetition of every check is asserted.